
Stateful Fuzzing Analysis with AFLNet

Aprille J. Xi
ajxi@andrew.cmu.edu

1 Background

Expanding upon Grey Box fuzzing as we have learned in lectures, it's evident that for deep and difficult software vulnerabilities, fuzzing can provide a great advantage. However, for systems that are heavily sensitive to states, such as with network protocols (ex. for handshake/authentication interactions, or server-client interactions in general), traditional stateless fuzzers may not be able to reach certain bugs efficiently, and may never reach deeper server logic.

Stateful fuzzing must consider not only the format of individual messages but also the order in which messages must be sent to reach interesting functionality. A network service often rejects malformed requests by closing the connection or resetting to an initial state. As a result, the fuzzer must generate sequences rather than stand alone messages, and it must ensure that messages appear at appropriate stages of the interaction.

Thus, we introduce AFLNet, an extension to the stateless fuzzer AFL [1]. AFLNet works by first gathering real server-client interactions through captured network traffic. These serve as initial seeds for the fuzzing algorithm, and the client calls and server responses are then parsed into sequences, as stated before. This serves as the corpus. The target state selector chooses a target state s to fuzz, and the sequence selector randomly chooses a sequence M from the corpus. With the given target state s and sequence M , the sequence mutator then outputs a mutate sequence M' . This is done by splitting M into three sections, $m1$, $m2$, and $m3$, where $m1$ is the prefix of executed messages that leads to the target state s , $m2$ are all messages executed after $m1$ that still result in target state s , and $m3$ are all messages left over. Stacked mutations, such as additions and deletions (similar to all the mutations we discussed in lecture), are then applied onto $m2$ to generate M' . These mutated sequences are then fed and run through the server, obtaining server responses based on M' . These responses are then sent to update the state machine, where the state machine's nodes are the states, and edges are transitions between states based on the executed messages. This server machine is then used in the target state selector.

The state is defined in AFLNet as simply the server response code, i.e. given the response "200 OK", the state the system is within is 200. Although this approach is attractive because it does not require manual modeling, it is simplistic. A single response code can represent many distinct situations. In LightFTP for example the numeric code 550 can correspond to unavailable files, denied permissions, resource exhaustion, aborted transfers, or ongoing conflicting actions. These represent different real server behaviors. They exercise different internal logic and may expose different vulnerabilities, but AFLNet merges them into a single abstract state. This can distort the inferred state machine, confuse the scheduler, and make it less likely that the fuzzer discovers deeper execution paths.

This project explores the tradeoff between greater state precision, and resource consumption.

2 Methodology

The project goal was to refine AFLNet without altering its basic structure, creating AFLNet+. I modified the state definition so that instead of extracting only the numeric prefix, the system collects the entire server response text such as "Permission denied" or "Data channel was closed by ABOR command from client." This is then hashed and appended after the server response code, such that if

two responses have different numeric prefixes, then there is a guarantee of different states, but if they have the same code and different texts, then there is a possibility of hash collisions (though this is not applicable in this project, as LightFTP does not have enough varying response texts per response code). This is the only modification to AFLNet. All other behaviors including mutation routines, sequence splitting logic, scheduling, and coverage instrumentation remain unchanged, ensuring that observed differences in fuzzing behavior arise from improved state modeling alone.

The evaluation environment uses LightFTP within the ProFuzzBench framework. For each configuration AFLNwe, AFLNet, and AFLNet+ four fuzzers run in parallel for eight hours. All tests use the same compilation flags, seed corpus, and harness configuration (which are given in the tutorial for LightFTP within ProFuzzBench’s README.md). Metrics include total executed sequences, execution rate, mutation depth, stability, number of paths discovered, favored paths, statement coverage, branch coverage, and state coverage. These statistics are taken from AFLNet’s generated logs and averaged across runs.

3 Results

We obtain the following results:

Table 1: Comparison of AFLNet, AFLNet+, and AFLNwe

Metric	AFLNet	AFLNet+	AFLNwe
cycles_done	199.50	206.00	115.50
execs_done	296257.75	293697.00	1489667.25
execs_per_sec	10.05	9.77	51.92
paths_total	479.75	468.25	69.25
paths_found	477.75	466.25	67.25
paths_favored	44.50	62.75	42.25
max_depth	8.75	7.50	7.25
stability	52.38	62.34	77.39
statement_cvg	63.98	63.52	35.80
branch_cvg	44.27	43.62	20.70
state_cvg	23.00	67.75	0

Table 2: AFLNet vs AFLNet+ Comparison

Metric	AFLNet	AFLNet+	Diff	% Change
cycles_done	199.50	206.00	+6.50	+3.3%
execs_done	296257.75	293697.00	-2560.75	-0.9%
execs_per_sec	10.05	9.77	-0.28	-2.8%
paths_total	479.75	468.25	-11.50	-2.4%
paths_found	477.75	466.25	-11.50	-2.4%
paths_favored	44.50	62.75	+18.25	+41.0%
max_depth	8.75	7.50	-1.25	-14.3%
stability	52.38	62.34	+9.97	+19.0%
statement_cvg	63.98	63.52	-0.45	-0.7%
branch_cvg	44.27	43.62	-0.65	-1.5%
state_cvg	23.00	67.75	+44.75	+194.6%

Table 3: AFLNwe vs AFLNet Comparison

Metric	AFLNwe	AFLNet	Diff	% Change
cycles_done	115.50	199.50	+84.00	+72.7%
execs_done	1489667.25	296257.75	-1193409.50	-80.1%
execs_per_sec	51.92	10.05	-41.87	-80.6%
paths_total	69.25	479.75	+410.50	+592.8%
paths_found	67.25	477.75	+410.50	+610.4%
paths_favored	42.25	44.50	+2.25	+5.3%
max_depth	7.25	8.75	+1.50	+20.7%
stability	77.39	52.38	-25.02	-32.3%
statement_cvg	35.80	63.98	+28.18	+78.7%
branch_cvg	20.70	44.27	+23.57	+113.9%
state_cvg	0.00	23.00	+23.00	N/A

4 Discussion and Analysis

We first see that AFLNet and AFLNet+ both reflect the AFLNet paper’s results when compared to AFLNwe, where the overhead is substantially greater, but the number of paths found as well as the coverage as a whole is greatly increased as well.

The richer state model has a noticeable effect on how AFLNet+ guides exploration. The number of favored paths, as expected, rises by approximately forty one percent. Favored paths are sequences that the scheduler prioritizes whenever they reach states or branches that are rare or otherwise promising. The dramatic increase in favored paths reflects the fact that the more detailed state model produces many additional discovery points that AFLNet had previously collapsed. With a greater number of abstract states in the protocol machine AFLNet+ labels a larger portion of the corpus as especially valuable. This leads the fuzzer to preserve and explore more of the generated sequences.

State precision also improves determinism. Stability, a measure of the consistency with which a fuzzer can reenter a state when replaying the same test case, increases by nineteen percent with AFLNet+. This improvement arises because the new state identifiers reduce ambiguity in the inferred protocol transitions. When different error conditions are no longer collapsed into a single numeric code the scheduler can return to the same state far more reliably.

At the same time the results show modest declines in certain traditional fuzzing metrics. Statement coverage decreases slightly by less than one percent, and branch coverage shows a similarly small reduction. There is also a decrease of approximately fourteen percent in the maximum mutation depth reached during execution. These declines do not reflect any loss in program exploration ability. Instead they reveal a common tension in state modeling. The introduction of many new distinctions among states causes the scheduler to distribute its effort more widely across the expanded state space. Some of these distinctions stem from slight differences in response text that do not correspond to meaningfully different control flow paths in the program. As a result the fuzzer spends some portion of its time exercising states that do not lead to deeper code regions. The reduced maximum depth therefore reflects a dilution of focus rather than a structural limitation of the new model.

Execution throughput remains largely unchanged. The number of completed cycles increases slightly while the total number of executed test cases decreases by less than one percent. These variations fall within normal noise levels for large scale fuzzing campaigns and are consistent with the small bookkeeping cost added by the more expressive state model.

Overall the quantitative results demonstrate two principal findings. First the original AFLNet approach that relies exclusively on numeric response codes significantly underrepresents the complexity of real server behavior. Second a more detailed state model can guide exploration more accurately and more deterministically even if the immediate gains in raw code coverage are modest. The observed tradeoff highlights an important research question regarding the design of adaptive state abstractions that can preserve meaningful distinctions without overwhelming the scheduler.

Conclusion

The introduction of a more precise state abstraction into AFLNet reveals both the promise and the limitations of state driven guidance in greybox fuzzing. By enriching AFLNet's view of the protocol state space, AFLNet+ encourages the fuzzer to explore a broader set of execution paths that correspond to finer distinctions in server behavior. This expansion substantially increases state coverage, which may be especially valuable for network protocols whose correctness depends heavily on the maintenance of a consistent session state. In these cases a detailed understanding of protocol responses provides the fuzzer with an improved ability to revisit and mutate important behavioral transitions.

The richer state model also contributes to greater stability during fuzzing. When the state identification mechanism captures server behavior more precisely, the fuzzer is able to reenter the same protocol state more reliably, which leads to a noticeably more deterministic exploration process. This is particularly important for stateful fuzzing, because effective mutation requires the ability to return to previously discovered states to search for deeper or more subtle transitions.

At the same time the results show that increasing the precision of the state model introduces tradeoffs. Some distinctions among protocol responses are not equally meaningful for program exploration, and treating all such distinctions as equally important can bias the scheduler toward states that do not correspond to new opportunities for code coverage. The increased granularity also introduces additional computational overhead, and the greater emphasis on state exploration can reduce the attention given to control flow based coverage, which remains central to the discovery of memory safety vulnerabilities and similar classes of bugs.

Overall the findings suggest that a more expressive state model can be beneficial, but only when the additional distinctions reflect meaningful differences in how the program processes input. The challenge therefore lies in striking an appropriate balance between protocol level understanding and the fundamental coverage driven principles that guide greybox fuzzing. Future work should explore adaptive techniques that allow the fuzzer to refine or merge states dynamically based on their actual influence on control flow exploration. Such an approach may allow fuzzers like AFLNet to retain the advantages of precise state modeling without incurring unnecessary overhead or sacrificing depth in the code paths that ultimately determine vulnerability discovery.

References

[1] V. -T. Pham, M. Böhme and A. Roychoudhury, "AFLNET: A Greybox Fuzzer for Network Protocols," 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), Porto, Portugal, 2020, pp. 460-465, doi: 10.1109/ICST46399.2020.00062. keywords: Servers;Protocols;Fuzzing;Tools;Data models;Security;Computer bugs,